

KarScan – Bash-Based Security Assessment Tool (`karscan.sh`)

A modular, interactive security assessment framework written in pure Bash. Given a target, it walks through reconnaissance, port & service enumeration, service-specific automated checks, and produces a structured, evidence-based report – asking the operator how to approach each stage instead of silently guessing.

Built for the **Instant Software Solutions Security Track** (Penetration Testing Diploma), assignment `SEC-BASH-092226`.

>  ****Only run this against hosts you own or are explicitly authorized to test**** – a local VirtualBox VM on a host-only network, or an authorized lab environment. Never point it at a public IP or live site without permission.

Why it's not "a one-liner around nmap"

`karscan.sh` uses `nmap` for the raw scanning primitive (as any real tool would), but everything around it – deciding what to run, parsing the results, interpreting service banners, deciding which checks apply, collecting evidence, and building the report – is Bash logic:

- Structured parsing of `nmap`'s greppable output into per-port service/version data (no scraping human-readable text).
- A decision layer that maps each discovered service to the right check module (`FTP` → `check\_ftp`, `SMB` → `check\_smb`, etc.) and only runs checks that apply to what was actually found.
- An ****interactive menu system**** (`ask\_menu` / `ask\_yes\_no` in `lib/utils.sh`) that lets the operator choose the recon depth, scan type, which service checks to run, and the report format – before each stage – while still supporting a fully non-interactive `--auto` / batch mode for scripting.
- Every finding is logged with concrete ****evidence**** (the exact command and response that triggered it) and a ****recommendation**** – never a bare label.

Features

- ****Reconnaissance**** – reachability check, IP/hostname resolution, local route & ARP info, optional traceroute and whois, all logged to `scan.txt`.
- ****Port & service enumeration**** – choice of fast/full/custom/SYN/connect scans, structured port → service → version parsing.
- ****Pre-scan profiling stage**** – before the port scan, chooses:
 - ****Firewall evasion****: optionally switches to a TCP Xmas scan (`-sX`) instead of a normal scan.
 - ****Protocol****: TCP or UDP (`-sU`), swapped in automatically.
 - ****Operational profile****: ***Penetration Test*** (standard `-T3` scan, exactly the original behavior) or ***Red Team*** (`-T1` paranoid timing, packet fragmentation `-f`, `--randomize-hosts`, padded packet length) for maximum practical stealth. The exact technique and flags used are always written out in full in `scan.txt`, `summary.txt`, and `report.html` – nothing is hidden from the report even when it's hidden from the target.
- ****Automated, decision-driven enumeration**** – checks only run for services that were actually detected:

```
| Service | Check |
|---|---|
| FTP | Anonymous login check |
| SSH | Version/banner check (flags outdated OpenSSH banners) |
| SMB | Share enumeration (`smbclient` / `enum4linux` / nmap fallback) |
| SMTP | Open relay probe + VRFY-based user enumeration |
| DNS | Zone transfer (AXFR) check |
| HTTP/HTTPS | Headers, missing security headers, `robots.txt`, common paths |
```

- ****Evidence-based reporting**** – `scan.txt`, `findings.txt`, `summary.txt`, and (optional) a styled `report.html`, all under `reports/<target>\_<timestamp>/`.
- ****Robust error handling**** – missing target, missing dependencies, unreachable host, and exit codes for each case (see below).
- ****Batch mode**** – `./karscan.sh -f targets.txt` scans a whole list with one report per host.
- ****`--help` / `--version` flags.****
- ****Fully modular architecture**** – one file per concern under `lib/` and `modules/`, sourced by `karscan.sh`.

Usage

```
```bash
chmod +x karscan.sh
./karscan.sh <target> # interactive single-target scan
./karscan.sh --auto <target> # non-interactive, safe defaults
./karscan.sh -f targets.txt # batch scan, one report per host
./karscan.sh --help
./karscan.sh --version
```
```

Example session:

```
...
$ ./karscan.sh 192.168.56.105

[?] Recon depth – how thorough should reconnaissance be?
  [1] Quick   – reachability + hostname resolution only
  * [2] Standard – quick + traceroute + local ARP/route info (recommended)
  [3] Deep    – standard + whois lookup
  [4] Skip recon entirely
  Choice [default 2]: 2

[?] Port scan type – how should we scan 192.168.56.105?
  * [1] Fast   – top 100 ports, service/version detection (recommended default)
  [2] Full    – all 65535 TCP ports, service/version detection (slow)
...
...

```

Project layout

```
...
KarScan/
├── karscan.sh           # entry point / orchestrator
├── lib/
│   ├── colors.sh       # terminal colors & UI helpers
│   ├── logging.sh      # log_ok/info/warn/fail/finding + log file
│   └── utils.sh        # dependency checks, validation, interactive menus
├── modules/
│   ├── recon.sh        # Stage 1: reconnaissance
│   ├── evasion.sh      # Pre-scan: firewall evasion / protocol / stealth profile
│   ├── portscan.sh     # Stage 2: port & service enumeration
│   ├── ftp.sh / ssh.sh / smb.sh / smtp.sh / dns.sh / http.sh # Stage 3: checks
│   └── report.sh       # Stage 4: scan.txt / findings.txt / summary.txt / report.html
└── reports/           # generated per-run, gitignored except .gitkeep
...

```

Error handling

| Condition | Behavior | Exit code |
|---|---|-----------|
| No target given | Prints usage | `1` |
| Required tool missing (`nmap`, `ping`, `curl`, `dig`) | Reports which and how to install | `2` |
| Target unreachable | Logs failure, still writes a partial report | `3` |

| User interrupts (Ctrl+C) | Clean message, no partial writes left dangling | `130` |

Setting up a target VM (VulnHub)

1. Download the VM image (`.ova`/`.zip` with `.vmdk`/`.vdi`) from vulnhub.com.
2. VirtualBox → **File → Import Appliance**, or create a new VM pointing at the existing disk.
3. **Settings → Network → Adapter 1 → Attached to: Host-only Adapter** – keep it isolated from your host network.
4. Start the VM, find its IP (`ip addr show` inside the VM, or `arp-scan --localnet` / `netdiscover -r 192.168.56.0/24` from the host).
5. Confirm connectivity: `ping -c 3 <VM\_IP>` before scanning.

What I implemented myself

All Bash logic – argument parsing, the interactive menu system, `nmap` output parsing, the service-check dispatch table, every check module, and the report generator – was written from scratch for this assignment. `nmap`, `smbclient`/`enum4linux`, `curl`, and `dig` are used as the underlying scanning primitives the checks are built on top of.

Disclaimer

For authorized security testing and educational use only. Only run against systems you own or have explicit written permission to test.